



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Introducción a la Programación

Guía de instalación

Fecha de compilación: 19/08/2025

Índice

Introducción	3
Visual Studio Code	4
1. Windows y macOS	4
2. Linux	4
Distribuciones basadas en Debian (Ubuntu, Linux Mint, etc.)	4
Distribuciones basadas en Fedora (Red Hat Enterprise Linux, etc.)	4
Distribuciones basadas en Arch Linux (EndeavourOS, CachyOS, etc.)	4
Haskell	5
1. Windows	5
GHCup (recomendado)	5
Windows Subsystem for Linux (avanzado)	11
Chocolatey	11
2. Linux	12
Distribuciones basadas en Debian (Ubuntu, Linux Mint, etc.)	12
Distribuciones basadas en Fedora (Red Hat Enterprise Linux, etc.)	12
Distribuciones basadas en Arch Linux (EndeavourOS, CachyOS, etc.)	12
3. macOS	12
4. Extensiones para Visual Studio Code	13
5. Uso de bibliotecas en un proyecto	13
HUnit	13
Python	14
1. Windows	14
Instalación por la página de Python (recomendado)	14
Windows Subsystem for Linux (avanzado)	17
Chocolatey	17
2. Linux	18
Distribuciones basadas en Debian (Ubuntu, Linux Mint, etc.)	18
Distribuciones basadas en Fedora (Red Hat Enterprise Linux, etc.)	18
Distribuciones basadas en Arch Linux (EndeavourOS, CachyOS, etc.)	18
3. macOS	18
4. Extensiones para Visual Studio Code	19
5. Uso de bibliotecas en un proyecto	20
pip	20
Entornos virtuales (venv) (recomendado)	21
Bibliotecas distribuidas por quienes mantienen la distribución de Linux	22
Troubleshooting	23
1. Me olvidé de agregar python.exe al PATH en Windows	23
2. Me olvidé de deslimitar los caracteres para direcciones largas en Windows	24
3. No tengo una computadora disponible donde instalar lo necesario para realizar la materia	25
Glosario	26

Introducción

Este documento está pensado como guía para los alumnos de la materia **Introducción a la Programación** en la instalación y configuración de los programas que necesitarán a lo largo de la cursada.

La idea es que, incluso sin conocimientos previos, puedan seguir los pasos sin dificultad y tener su entorno listo para programar. La guía cubre la instalación de *Python*, *Haskell* y otras herramientas necesarias, explicando las opciones más recomendadas y los posibles problemas que pueden surgir durante el proceso. **Sigan únicamente las instrucciones correspondientes a su sistema operativo.**

Además, incluimos una sección de [Troubleshooting](#) para resolver inconvenientes comunes, y un [Glosario](#) al final, con explicaciones sencillas de los términos técnicos que aparecen en el documento, para que puedan comprender cada concepto sin necesidad de buscarlos por su cuenta.

Ante cualquier duda o inconveniente, consulten a los docentes de la materia. Aun así, fomentamos que intenten resolverlo por su cuenta buscando en internet, leyendo foros, la documentación oficial, mirando videos o preguntándole a su LLM de confianza.

Visual Studio Code

Recomendamos Visual Studio Code (VSCode) como editor gratuito y multiplataforma para programar en Haskell y Python.

1. Windows y macOS

Deben dirigirse a la página oficial de [Visual Studio Code](#) y descargar el instalador correspondiente a su sistema operativo. Se instala como cualquier aplicación de Windows o macOS.

2. Linux

Generalmente, Visual Studio Code se encuentra en los repositorios de las distribuciones de Linux, por lo que, **antes de ejecutar cualquiera de los comandos listados a continuación**, es recomendable **abrir la tienda de aplicaciones**, centro de aplicaciones, etc., de su distribución y **buscar «Visual Studio Code»** para instalarlo desde ahí, si está disponible.

Pueden encontrar información más detallada en [este enlace](#).

Distribuciones basadas en Debian (Ubuntu, Linux Mint, etc.)

1. Agreguen el repositorio de Microsoft a su instalación de Debian:

```
echo "code code/add-microsoft-repo boolean true" | sudo debconf-set-selections
```

2. Instalen apt-transport-https, actualicen la caché de sus paquetes e instalen Visual Studio Code:

```
sudo apt install apt-transport-https
sudo apt update
sudo apt install code
```

Distribuciones basadas en Fedora (Red Hat Enterprise Linux, etc.)

1. Agreguen el repositorio de Microsoft a su instalación de Fedora:

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
echo -e "[code]\nname=Visual Studio Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\nautorefresh=1\ntype=rpm-md\nngpgcheck=1\nngpgkey=https://packages.microsoft.com/keys/microsoft.asc" |
sudo tee /etc/yum.repos.d/vscode.repo > /dev/null
```

2. Actualicen la lista de paquetes e instalen Visual Studio Code:

```
dnf check-update
sudo dnf install code
```

Distribuciones basadas en Arch Linux (EndeavourOS, CachyOS, etc.)

1. Es recomendable instalar algún AUR helper, como `paru` o `yay`, si no lo tienen instalado. Pueden utilizar `paru --version` y `yay --version` para verificar si poseen alguno de estos instalados.
2. Pueden instalar `paru` con los siguientes comandos:

```
sudo pacman -S git
git clone https://aur.archlinux.org/paru-bin.git && cd paru-bin
makepkg -si
```

3. Luego, pueden instalar Visual Studio Code con el siguiente comando:

```
paru -S visual-studio-code-bin
```

Haskell

Tras la instalación de Haskell en Windows o macOS, es importante cerrar sus terminales abiertas y/o instancias de Visual Studio Code (o su editor de preferencia) para que los cambios en las variables de entorno surtan efecto; de lo contrario, no podrán ejecutar `ghc`, `ghci`, `cabal` y otras herramientas asociadas.

1. Windows

Hay varias formas de instalar el intérprete de Haskell en Windows. A continuación, presentamos algunos métodos donde tuvimos buenas experiencias.

Deben elegir solo una.

GHCup (recomendado)

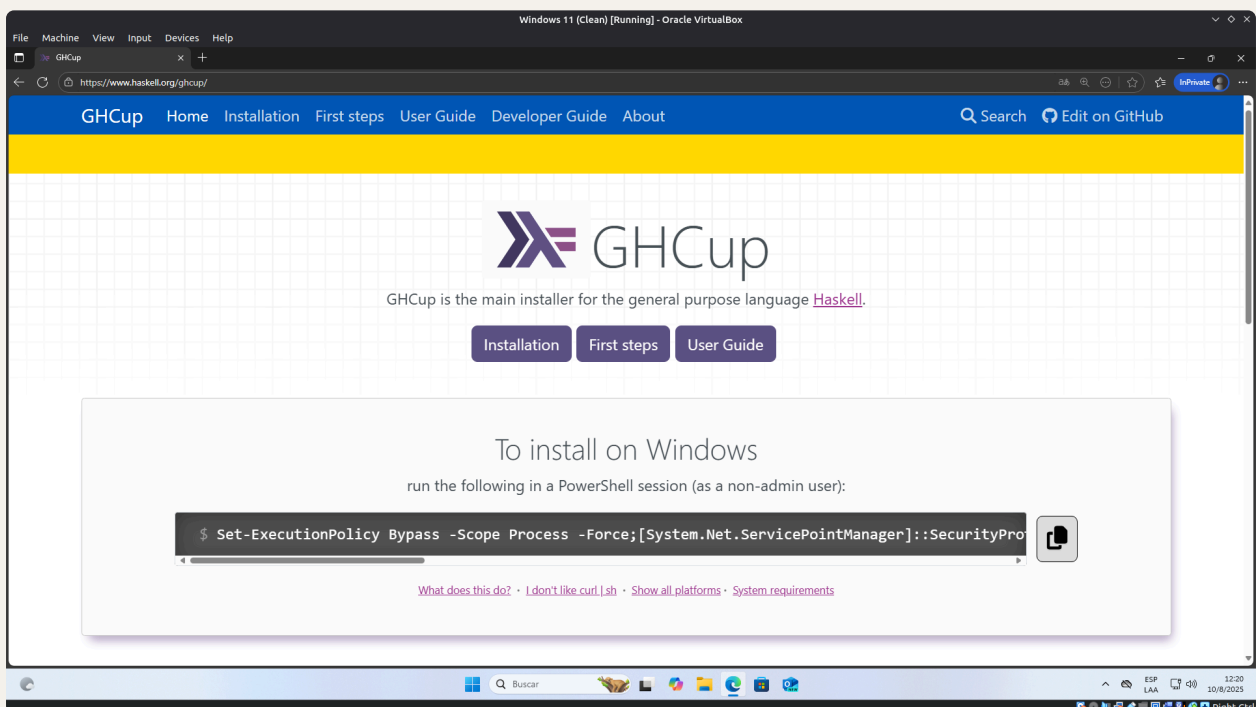
Si ya instalaste Haskell usando alguno de los métodos que aparecen más abajo, no es necesario que sigas este procedimiento.

Este es el método recomendado por el equipo de desarrollo detrás de Haskell.

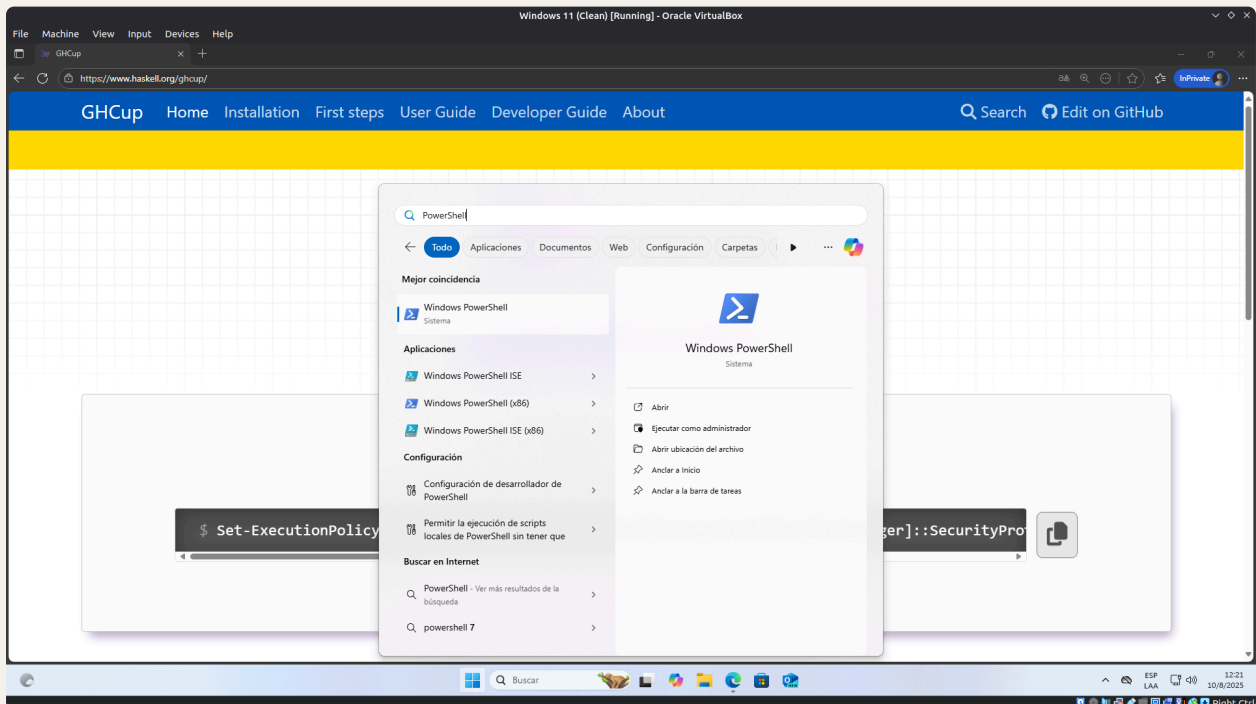
Deben ingresar a la página de [GHCup](https://www.haskell.org/ghcup/) y seguir las instrucciones de la página, luego del instalador. Al finalizar la instalación deben **cerrar la terminal actual**, ya que el instalador modifica algunas variables de entorno que no se aplicarán en la terminal abierta.

A continuación les dejamos un resumen de los pasos a seguir:

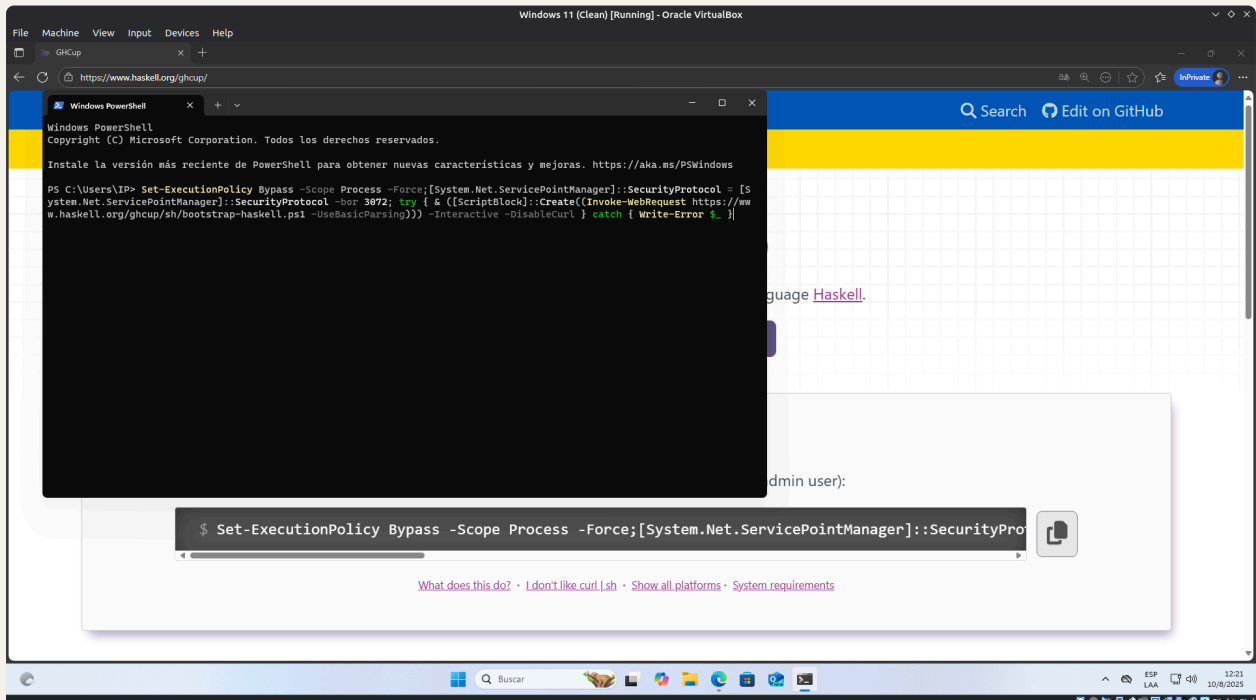
1. Vayan a la página oficial de [GHCup](https://www.haskell.org/ghcup/) y copien el comando de instalación que aparece en el centro de la página haciendo clic izquierdo en el botón que contiene el ícono de un archivo.



2. Abran el buscador de Windows y escriban **PowerShell**, es importante que **no lo ejecuten como administrador**. **Simplemente deben darle clic izquierdo para abrirlo**. Queremos abrir el programa que se ve en la imagen, con el nombre e ícono que se ve a continuación:

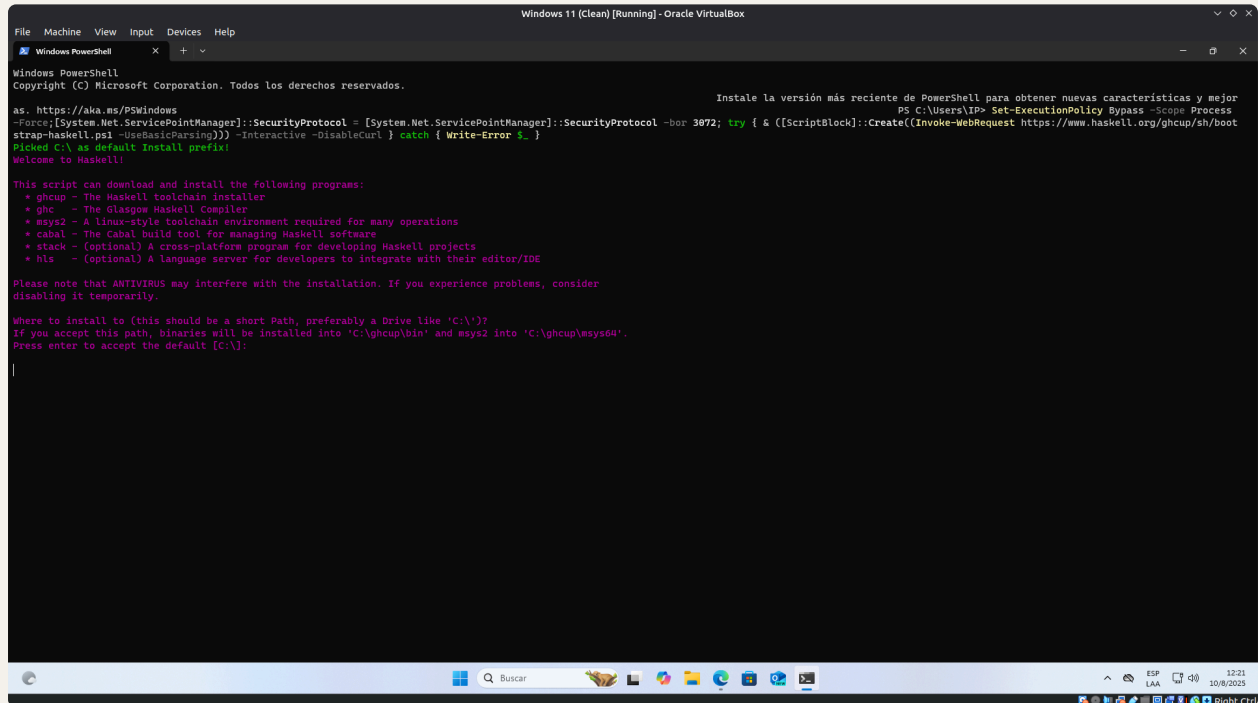


3. Una vez abierto **PowerShell**, **peguen** el comando que copiaron de la página con **clic derecho** y presionen **Enter**.



4. El instalador les hará una serie de preguntas, lo más recomendable es apretar **Enter** para aceptar las opciones predeterminadas, a menos que tengan una razón específica para cambiar alguna de ellas. Si quieren investigar qué hace cada opción son libres de hacerlo, para la materia basta con aceptar las opciones predeterminadas.

5. Pregunta 1:



```
File Machine View Input Devices Help
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

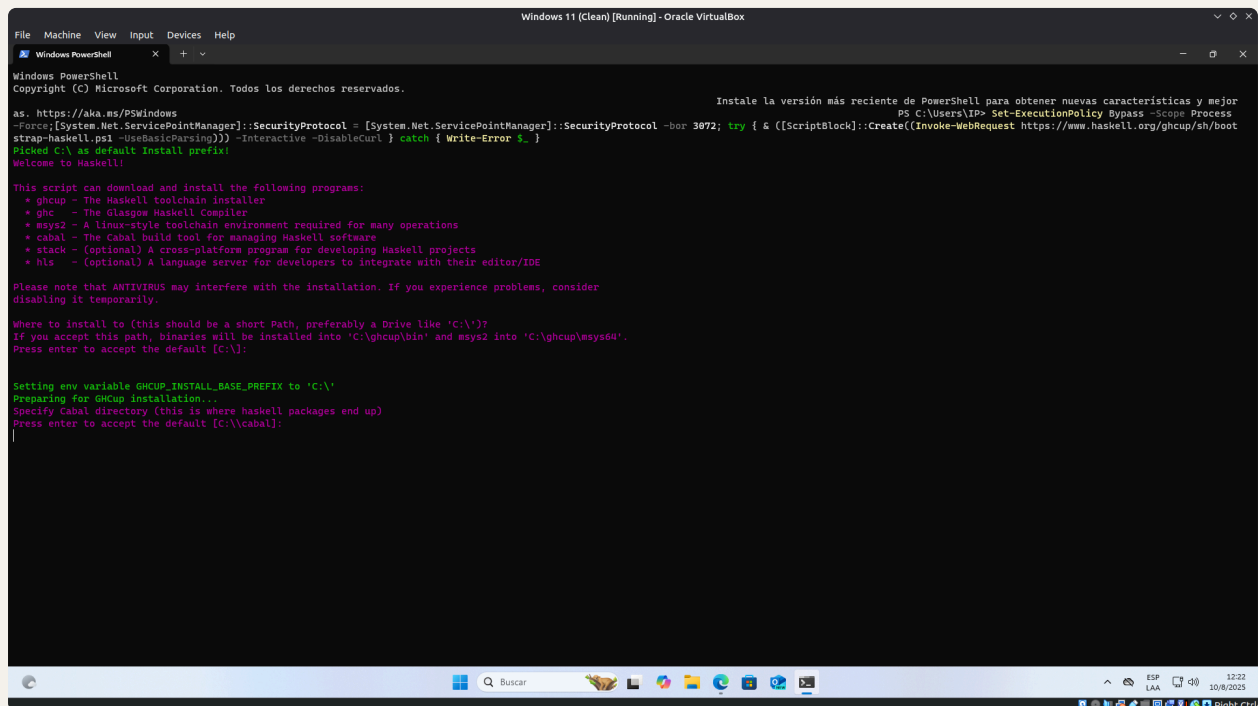
Instale la versión más reciente de PowerShell para obtener nuevas características y mejor
PS C:\Users\IP> Set-ExecutionPolicy Bypass -Scope Process
-Force;[System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; try { & ([ScriptBlock]::Create((Invoke-WebRequest https://www.haskell.org/ghcup/sh/boot
strap-haskell.ps1 -useBasicParsing))) -Interactive -DisableCurl } catch { Write-Error $_ }
Picked C:\ as default Install prefix!
Welcome to Haskell!

This script can download and install the following programs:
* ghcup - The Haskell toolchain installer
* ghc - The Glasgow Haskell Compiler
* msys2 - A linux-style toolchain environment required for many operations
* cabal - The Cabal build tool for managing Haskell software
* stack - (Optional) A cross-platform program for developing Haskell projects
* hls - (Optional) A language server for developers to integrate with their editor/IDE

Please note that ANTI VIRUS may interfere with the installation. If you experience problems, consider
disabling it temporarily.

Where to install to (this should be a short Path, preferably a Drive like 'C:\')?
If you accept this path, binaries will be installed into 'C:\ghcup\bin' and msys2 into 'C:\ghcup\msys64'.
Press enter to accept the default [C:\]:
```

6. Pregunta 2:



```
File Machine View Input Devices Help
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejor
PS C:\Users\IP> Set-ExecutionPolicy Bypass -Scope Process
-Force;[System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; try { & ([ScriptBlock]::Create((Invoke-WebRequest https://www.haskell.org/ghcup/sh/boot
strap-haskell.ps1 -useBasicParsing))) -Interactive -DisableCurl } catch { Write-Error $_ }
Picked C:\ as default Install prefix!
Welcome to Haskell!

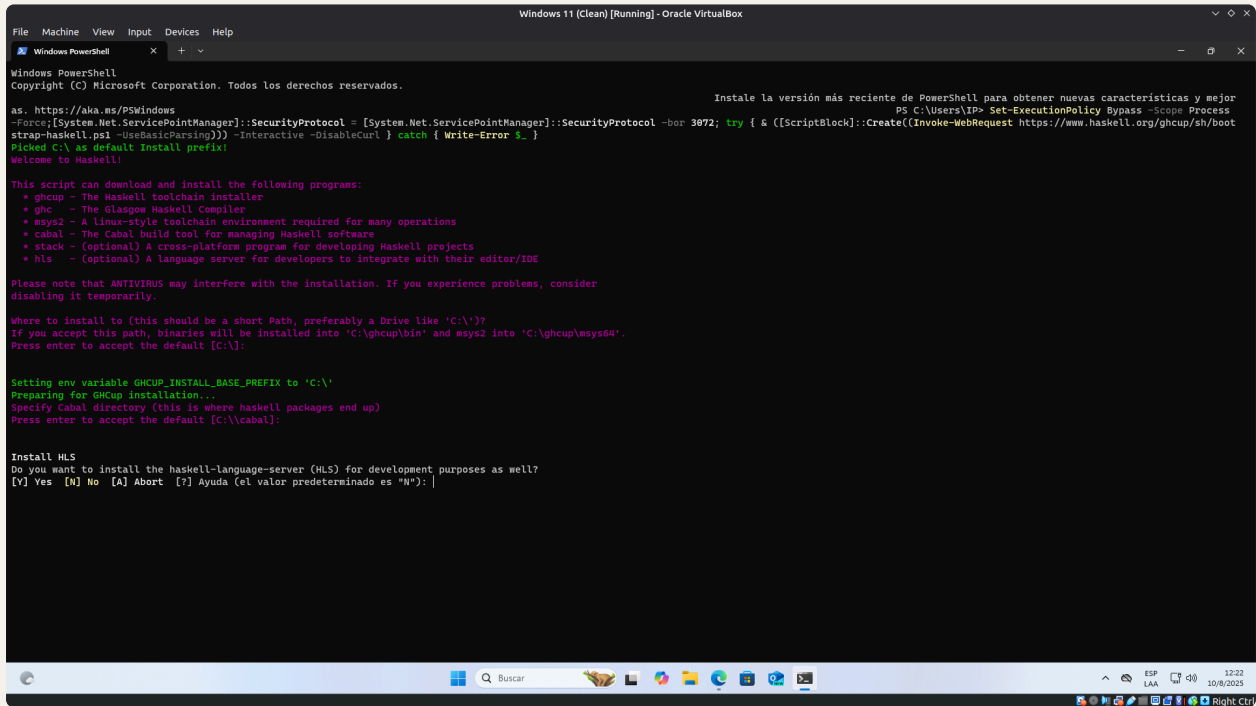
This script can download and install the following programs:
* ghcup - The Haskell toolchain installer
* ghc - The Glasgow Haskell Compiler
* msys2 - A linux-style toolchain environment required for many operations
* cabal - The Cabal build tool for managing Haskell software
* stack - (Optional) A cross-platform program for developing Haskell projects
* hls - (Optional) A language server for developers to integrate with their editor/IDE

Please note that ANTI VIRUS may interfere with the installation. If you experience problems, consider
disabling it temporarily.

Where to install to (this should be a short Path, preferably a Drive like 'C:\')?
If you accept this path, binaries will be installed into 'C:\ghcup\bin' and msys2 into 'C:\ghcup\msys64'.
Press enter to accept the default [C:\]:

Setting env variable GHUP_INSTALL_BASE_PREFIX to 'C:\'
Preparing for GHUP installation...
Specify Cabal directory (this is where haskell packages end up)
Press enter to accept the default [C:\cabal]:
```

7. Pregunta 3:



The screenshot shows a Windows 11 (Clean) [Running] - Oracle VirtualBox window. Inside, a Windows PowerShell terminal is open. The terminal displays the output of a script that installs Haskell tools. The script starts with a copyright notice and a message about installing the latest PowerShell version. It then lists the programs to be installed: ghcup (Haskell toolchain installer), ghc (Glasgow Haskell Compiler), msys2 (Linux-style toolchain environment), cabal (Cabal build tool), stack (cross-platform program for developing Haskell projects), and hls (language server for developers). The script prompts the user to accept the default installation path (C:\) and to specify a Cabal directory (C:\cabal). It then prompts the user to install HLS (Haskell Language Server) for development purposes. The user has responded with 'Y' for Yes.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejor
PS C:\Users\IP> Set-ExecutionPolicy Bypass -Scope Process
strap-haskell.ps1 -UseBasicParsing))) -Interactive -DisableCurl } catch { Write-Error $_ }
Picked C:\ as default Install prefix!
Welcome to Haskell!

This script can download and install the following programs:
* ghcup - The Haskell toolchain installer
* ghc - The Glasgow Haskell Compiler
* msys2 - A linux-style toolchain environment required for many operations
* cabal - The Cabal build tool for managing Haskell software
* stack - (optional) A cross-platform program for developing Haskell projects
* hls - (optional) A language server for developers to integrate with their editor/IDE

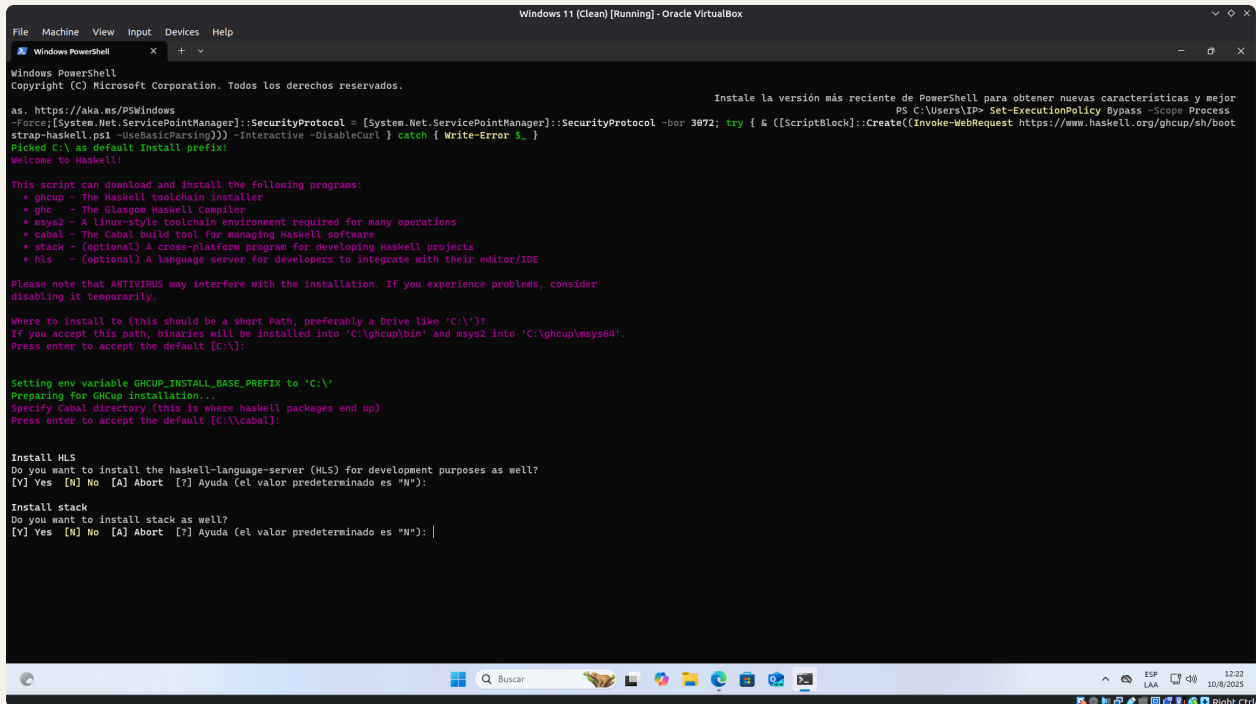
Please note that ANTIIVIRUS may interfere with the installation. If you experience problems, consider
disabling it temporarily.

Where to install to (this should be a short Path, preferably a Drive like 'C:\')?
If you accept this path, binaries will be installed into 'C:\ghcup\bin' and msys2 into 'C:\ghcup\msys64'.
Press enter to accept the default [C:\]:

Setting env variable GHCUPISTALLBASEPREFIX to 'C:\'
Preparing for GHCUPISTALL installation...
Specify Cabal directory (this is where haskell packages end up)
Press enter to accept the default [C:\cabal]:

Install HLS
Do you want to install the haskell-language-server (HLS) for development purposes as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"): |
```

8. Pregunta 4:



The screenshot shows a Windows 11 (Clean) [Running] - Oracle VirtualBox window. Inside, a Windows PowerShell terminal is open. The terminal displays the output of a script that installs Haskell tools. The script starts with a copyright notice and a message about installing the latest PowerShell version. It then lists the programs to be installed: ghcup (Haskell toolchain installer), ghc (Glasgow Haskell Compiler), msys2 (Linux-style toolchain environment), cabal (Cabal build tool), stack (cross-platform program for developing Haskell projects), and hls (language server for developers). The script prompts the user to accept the default installation path (C:\) and to specify a Cabal directory (C:\cabal). It then prompts the user to install HLS (Haskell Language Server) for development purposes. The user has responded with 'Y' for Yes.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejor
PS C:\Users\IP> Set-ExecutionPolicy Bypass -Scope Process
strap-haskell.ps1 -UseBasicParsing))) -Interactive -DisableCurl } catch { Write-Error $_ }
Picked C:\ as default Install prefix!
Welcome to Haskell!

This script can download and install the following programs:
* ghcup - The Haskell toolchain installer
* ghc - The Glasgow Haskell Compiler
* msys2 - A linux-style toolchain environment required for many operations
* cabal - The Cabal build tool for managing Haskell software
* stack - (optional) A cross-platform program for developing Haskell projects
* hls - (optional) A language server for developers to integrate with their editor/IDE

Please note that ANTIIVIRUS may interfere with the installation. If you experience problems, consider
disabling it temporarily.

Where to install to (this should be a short Path, preferably a Drive like 'C:\')?
If you accept this path, binaries will be installed into 'C:\ghcup\bin' and msys2 into 'C:\ghcup\msys64'.
Press enter to accept the default [C:\]:

Setting env variable GHCUPISTALLBASEPREFIX to 'C:\'
Preparing for GHCUPISTALL installation...
Specify Cabal directory (this is where haskell packages end up)
Press enter to accept the default [C:\cabal]:

Install HLS
Do you want to install the haskell-language-server (HLS) for development purposes as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"): |

Install stack
Do you want to install stack as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"): |
```


9. En esta pregunta, la pregunta 5, es posible que deseen cambiar la opción predeterminada de **[Y]** (Sí) a **[N]** (No). Para ello simplemente escriben **N** y presionan **Enter**.

```
File Machine View Input Devices Help
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejor
PS C:\Users\IP> Set-ExecutionPolicy Bypass -Scope Process
strap-haskell.ps1 -UseBasicParsing)) -Interactive -DisableCurl } catch { Write-Error $_ }
Picked C:\ as default Install prefix!
Welcome to Haskell!

This script can download and install the following programs:
* ghcup - The Haskell toolchain installer
* ghc - The Glasgow Haskell Compiler
* msys2 - A linux-style toolchain environment required for many operations
* cabal - The Cabal build tool for managing Haskell software
* stack - (Optional) A cross-platform program for developing Haskell projects
* hls - (Optional) A language server for developers to integrate with their editor/IDE

Please note that ANTIVIRUS may interfere with the installation. If you experience problems, consider
disabling it temporarily.

Where to install to (this should be a short Path, preferably a Drive like 'C:\')?
If you accept this path, binaries will be installed into 'C:\ghcup\bin' and msys2 into 'C:\ghcup\msys64'.
Press enter to accept the default [C:\]:

Setting env variable GHCUK_INSTALL_BASE_PREFIX to 'C:\'
Preparing for GHCUK installation...
Specify Cabal directory (this is where haskell packages end up)
Press enter to accept the default [C:\cabal]:

Install HLS
Do you want to install the haskell-language-server (HLS) for development purposes as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"):

Install stack
Do you want to install stack as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"):

Create Desktop shortcuts
Do you want to create convenience desktop shortcuts (e.g. for uninstallation and msys2 shell)?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "Y"): |
```

10. Pregunta 6:

```
File Machine View Input Devices Help
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejor
PS C:\Users\IP> Set-ExecutionPolicy Bypass -Scope Process
strap-haskell.ps1 -UseBasicParsing)) -Interactive -DisableCurl } catch { Write-Error $_ }
Picked C:\ as default Install prefix!
Welcome to Haskell!

This script can download and install the following programs:
* ghcup - The Haskell toolchain installer
* ghc - The Glasgow Haskell Compiler
* msys2 - A linux-style toolchain environment required for many operations
* cabal - The Cabal build tool for managing Haskell software
* stack - (Optional) A cross-platform program for developing Haskell projects
* hls - (Optional) A language server for developers to integrate with their editor/IDE

Please note that ANTIVIRUS may interfere with the installation. If you experience problems, consider
disabling it temporarily.

Where to install to (this should be a short Path, preferably a Drive like 'C:\')?
If you accept this path, binaries will be installed into 'C:\ghcup\bin' and msys2 into 'C:\ghcup\msys64'.
Press enter to accept the default [C:\]:

Setting env variable GHCUK_INSTALL_BASE_PREFIX to 'C:\'
Preparing for GHCUK installation...
Specify Cabal directory (this is where haskell packages end up)
Press enter to accept the default [C:\cabal]:

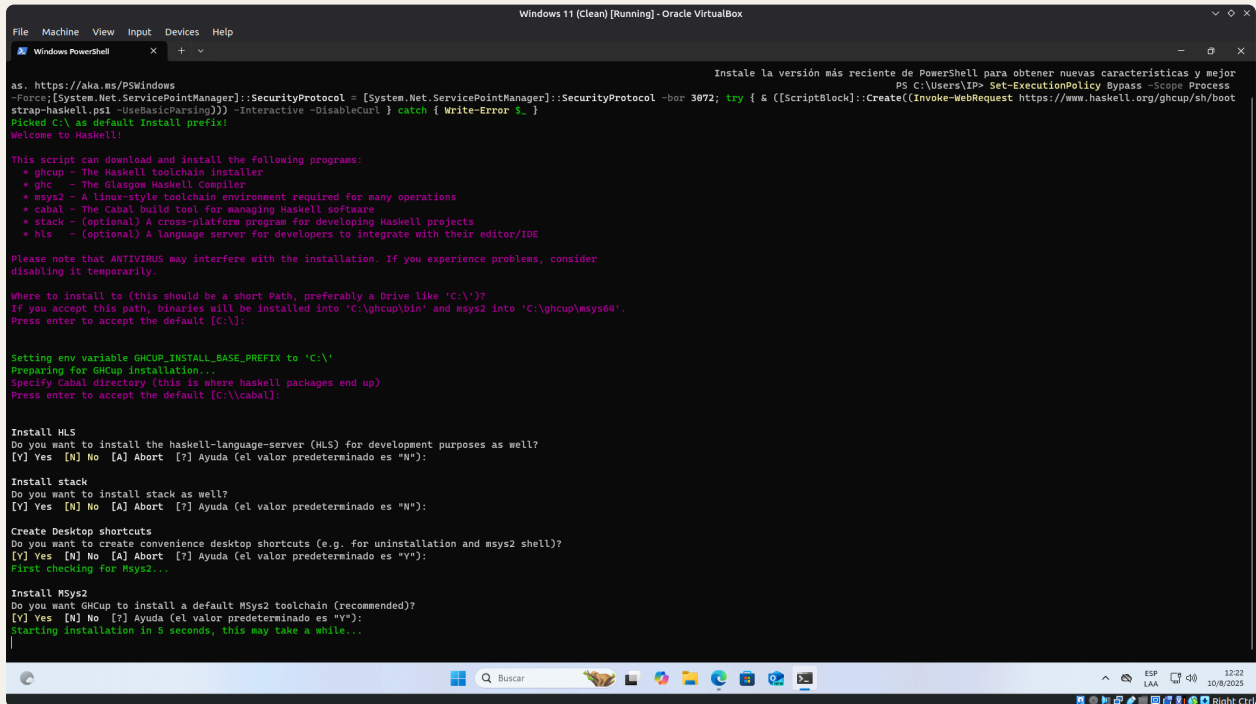
Install HLS
Do you want to install the haskell-language-server (HLS) for development purposes as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"):

Install stack
Do you want to install stack as well?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "N"):

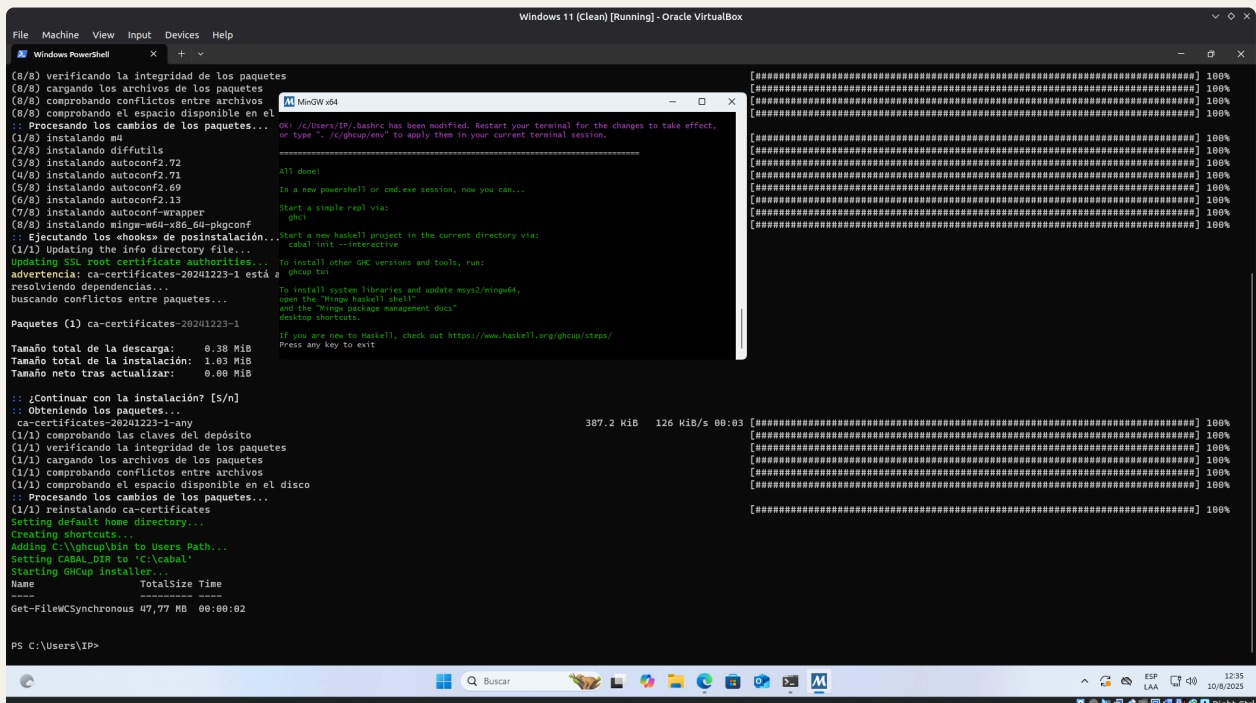
Create Desktop shortcuts
Do you want to create convenience desktop shortcuts (e.g. for uninstallation and msys2 shell)?
[Y] Yes [N] No [A] Abort [?] Ayuda (el valor predeterminado es "Y"): |
First checking for Msys2...

Install Msys2
Do you want GHCUK to install a default MSys2 toolchain (recommended)?
[Y] Yes [N] No [?] Ayuda (el valor predeterminado es "Y"): |
```

11. Tras la pregunta anterior -la pregunta 6- el instalador comenzará a descargar e instalar los componentes necesarios. Este proceso puede tardar un poco. Verán que en un momento de la instalación se les abrirá una nueva terminal, no se preocupen, es parte del instalador.



12. Una vez terminada la instalación se encontrarán con una pantalla similar a la siguiente:



- Deben cerrar ambas terminales, tanto la que abrió el instalador como la que usaron para ejecutar el comando de instalación.

Con esto ya deberían tener **Haskell** instalado en su sistema. Para verificar que la instalación fue exitosa, abran una nueva terminal y escriban `ghc --version`¹.

¹La primera vez que ejecuten el comando `ghc` puede tardar un poco.

Windows Subsystem for Linux (avanzado)

Si ya realizaste la instalación usando GHCup (opción de arriba) o Chocolatey (opción de abajo), no es necesario que sigas este método.

Si desean utilizar Haskell en un entorno Linux, pueden instalar el subsistema de Windows para Linux (WSL). Para ello, deben seguir las instrucciones de la [documentación oficial de Microsoft](#). Luego, deberán ir al apartado de [Linux](#) para seguir con la instalación.

Chocolatey

Si ya realizaste la instalación usando WSL (segunda opción) o GHCup (primera opción), no es necesario que sigas este método.

Deberán instalar el gestor de paquetes Chocolatey, para ello entren a [este link](#) y sigan las instrucciones de la página.

Recuerden que deben seleccionar la opción que dice «**Individual**».

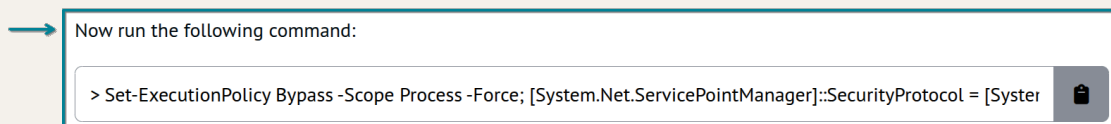
1. Choose How to Install Chocolatey:



Luego, copian y pegan el comando que les aparece en la página en **PowerShell**. Para copiarlo deben hacer clic en el botón que aparece a la derecha del comando, botón que contiene el ícono de un portapapeles.

Para abrir un **PowerShell** deben hacer lo siguiente:

1. Aprietan la tecla de Windows.
2. Escriben **PowerShell**.
3. Hacen clic derecho en la aplicación de nombre **PowerShell** que les aparece y seleccionan «**Ejecutar como administrador**».
4. Pegan el comando que **copiaron de la página** y aprietan Enter.



Una vez que se haya instalado Chocolatey, deben cerrar esa terminal de **PowerShell**; acto seguido abrir uno nuevo y ejecutar `choco install ghc cabal -y`.

2. Linux

El mundo de las distribuciones de Linux es **amplio y diverso**, y **la instalación de Haskell puede variar según la distribución que estén utilizando**.

Les mostramos cómo instalar Haskell en algunas de las distribuciones más populares. Para cada una de las distintas distribuciones deberán primero abrir una terminal, para luego, seguir los pasos correspondientes.

Distribuciones basadas en Debian (Ubuntu, Linux Mint, etc.)

1. Actualicen la caché de sus paquetes:

```
sudo apt update
```

2. Instalen GHC (el compilador de Haskell) y Cabal (el gestor de paquetes de Haskell):

```
sudo apt install ghc cabal-install
```

Distribuciones basadas en Fedora (Red Hat Enterprise Linux, etc.)

1. Actualicen la caché de sus paquetes:

```
sudo dnf update
```

2. Instalen GHC (el compilador de Haskell) y Cabal (el gestor de paquetes de Haskell):

```
sudo dnf install ghc cabal-install
```

Distribuciones basadas en Arch Linux (EndeavourOS, CachyOS, etc.)

1. Actualicen tanto la caché de sus paquetes como los paquetes instalados:

```
sudo pacman -Syu
```

2. Instalen GHC (el compilador de Haskell) y Cabal (el gestor de paquetes de Haskell):

```
sudo pacman -S ghc cabal-install
```

3. macOS

Deberán abrir una terminal y ejecutar el siguiente comando:

```
curl -k --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

El instalador realizará una serie de preguntas. Basta con seleccionar la opción por defecto en cada una de ellas, presionando **Enter**.

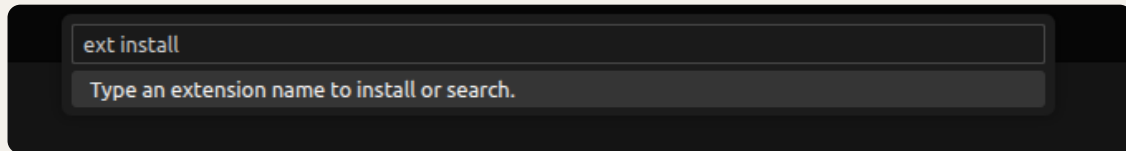
Al finalizar la instalación es importante **cerrar la terminal actual**, ya que el instalador modifica algunas variables de entorno que no se aplicarán en la terminal abierta.

4. Extensiones para Visual Studio Code

Una vez instalado Haskell, nos va a interesar instalar algunas extensiones a Visual Studio Code que nos ayudarán a mejorar nuestra experiencia de desarrollo.

Para ello, deben abrir Visual Studio Code y seguir estos pasos:

1. Aprieten `Ctrl + P` en Windows/Linux o `Command + P` en macOS para abrir la barra de comandos.



2. Escriban `ext install haskell.haskell` y presionen `Enter` para instalar la extensión oficial de Haskell.

Si quieren buscar extensiones adicionales, pueden abrir la vista de extensiones haciendo clic en el ícono de extensiones en la barra lateral izquierda o presionando `Ctrl + Shift + X` en Windows/Linux o `Command + Shift + X` en macOS. Desde allí, pueden buscar e instalar otras extensiones que les interesen. Pero tengan cuidado, la tienda de extensiones de Visual Studio Code no es *curada*, es decir, no todas las extensiones están auditadas; descargar una extensión de una fuente no confiable puede comprometer la seguridad de su sistema. Por lo tanto, asegúrense de instalar solo extensiones de fuentes confiables y revisadas por la comunidad.

5. Uso de bibliotecas en un proyecto

Distintos desarrolladores de software han creado bibliotecas² que pueden ser utilizadas en nuestros proyectos de Haskell. Estas bibliotecas nos permiten aprovechar el trabajo de otros y enfocarnos en la lógica de nuestro programa.

HUnit

Por ejemplo, en la materia utilizamos `HUnit` al realizar testing unitario. Para instalar una biblioteca en Haskell, utilizamos el gestor de paquetes `cabal`.

1. Primero, necesitamos actualizar la lista de paquetes disponibles. Abrimos una terminal y ejecutamos:

```
cabal update
```

2. Luego, para instalar una biblioteca específica, usamos el siguiente comando:

```
cabal install HUnit
```

Si no tienen instalado `cabal`, deben seguir las instrucciones de instalación de Haskell en su sistema operativo mencionadas anteriormente.

Para verificar que la instalación fue exitosa, pueden abrir una terminal y ejecutar `ghci`, el intérprete interactivo de Haskell, y luego intentar importar la biblioteca con `import Test.HUnit`. Si no hay errores, la instalación fue exitosa.

²Notar que «librerías» está mal dicho, la confusión viene de traducir erróneamente «library»/«libraries» del inglés.

Python

Tras la instalación de Python en Windows o macOS, es importante cerrar sus terminales abiertas y/o instancias de Visual Studio Code (o su editor de preferencia) para que los cambios en las variables de entorno surtan efecto; de lo contrario, no podrán ejecutar `python`, `pip` y otras herramientas asociadas.

1. Windows

Hay varias formas de instalar el intérprete de Python en Windows. A continuación, presentamos algunos métodos donde tuvimos buenas experiencias.

Deben elegir solo una.

Instalación por la página de Python (recomendado)

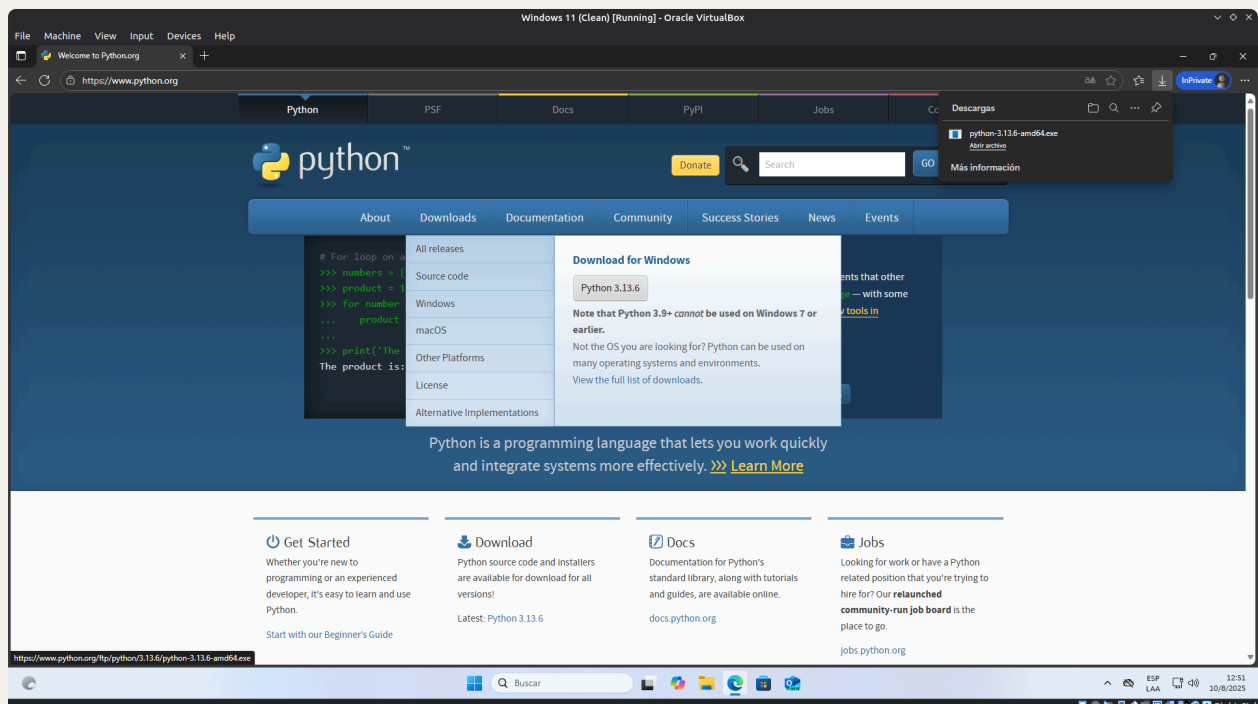
Si ya instalaste Python usando alguno de los métodos que aparecen más abajo, no es necesario que sigas este procedimiento.

Este es el método recomendado por el equipo de desarrollo detrás de Python.

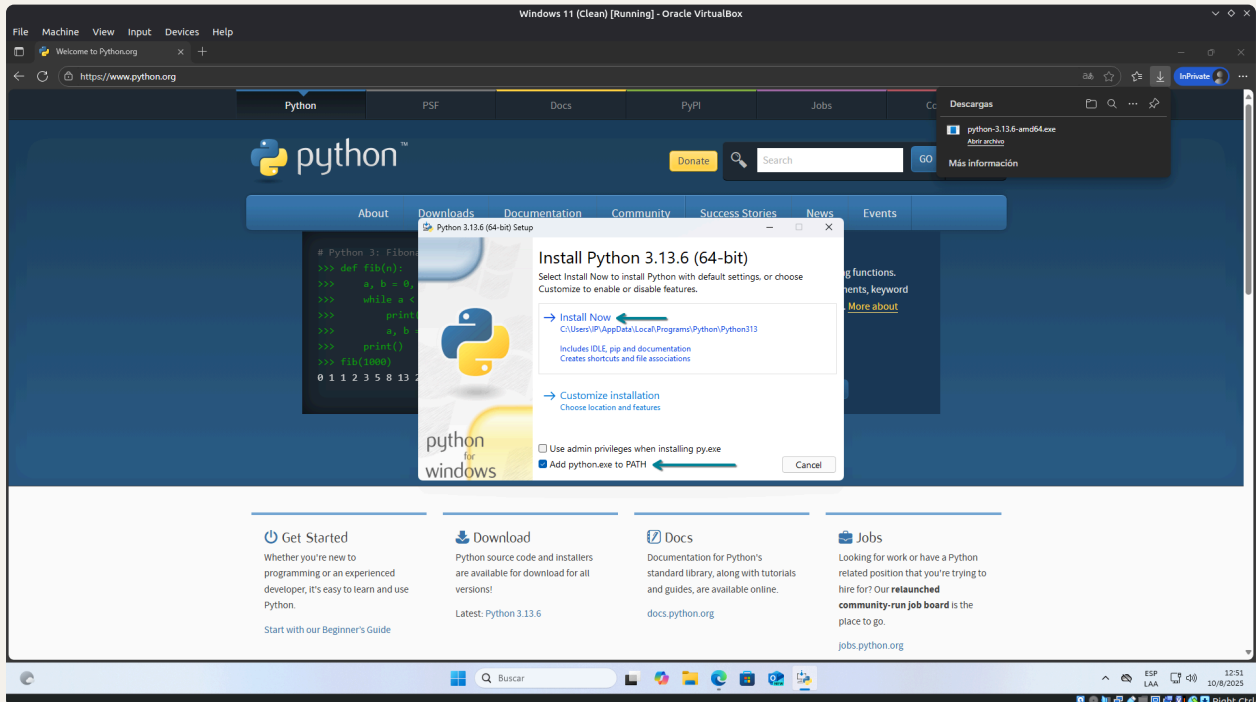
Deberán ingresar a la página de [Python](https://python.org) y descargar el instalador de la última versión de Python para Windows. Una vez descargado, lo ejecutan y siguen las instrucciones del mismo.

A continuación les dejamos un resumen de los pasos a seguir:

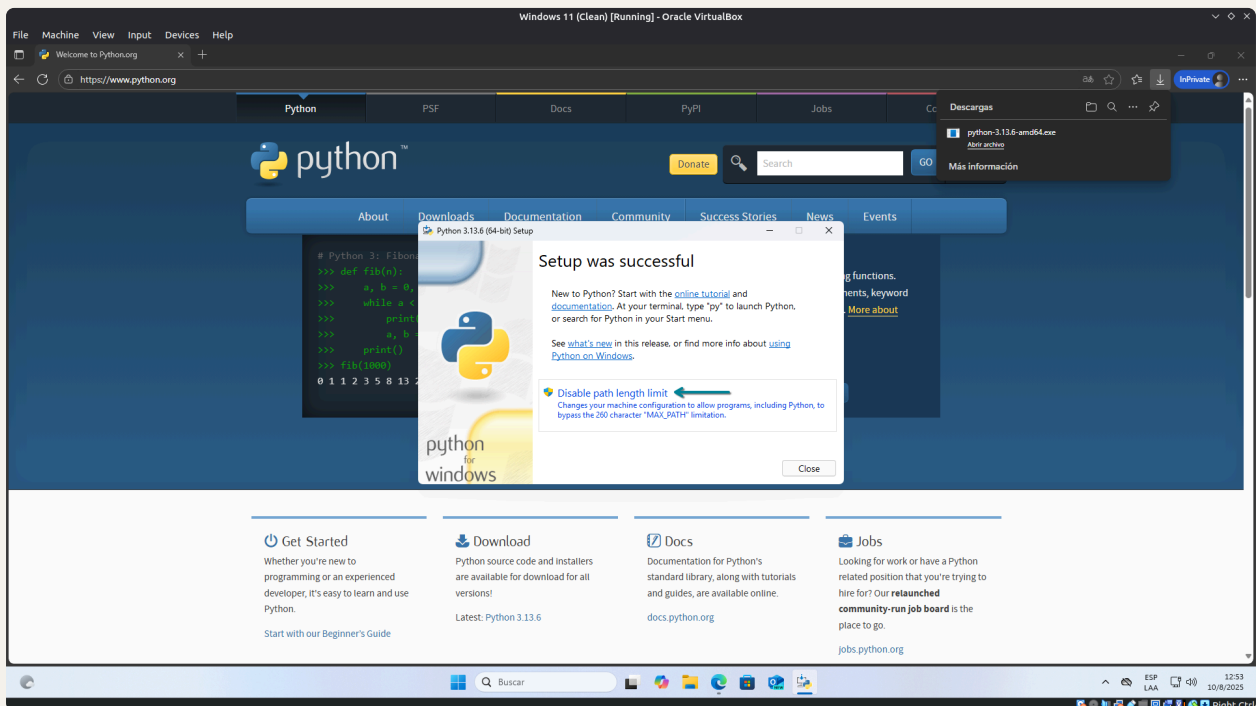
1. Vayan a la página oficial de [Python](https://python.org), pongan el mouse sobre el menú de «Downloads» y seleccionen la opción «Windows». Luego, hagan clic en el botón gris que dice «Python X.X.X» (donde X.X.X es la versión más reciente).



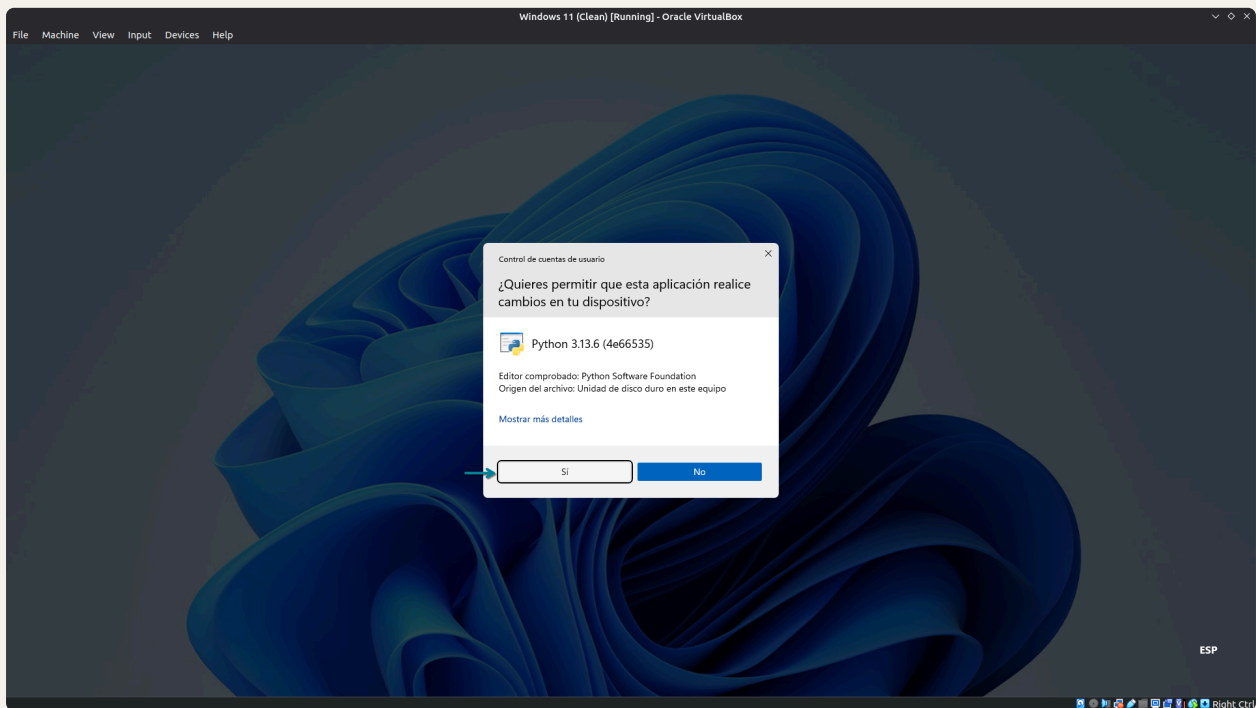
2. Una vez descargado el instalador de Python, ejecuten el archivo descargado. **Asegúrense de marcar la casilla que dice «Add python.exe to PATH» antes de hacer clic en «Install Now».** Esto es importante para que puedan ejecutar Python desde cualquier terminal.



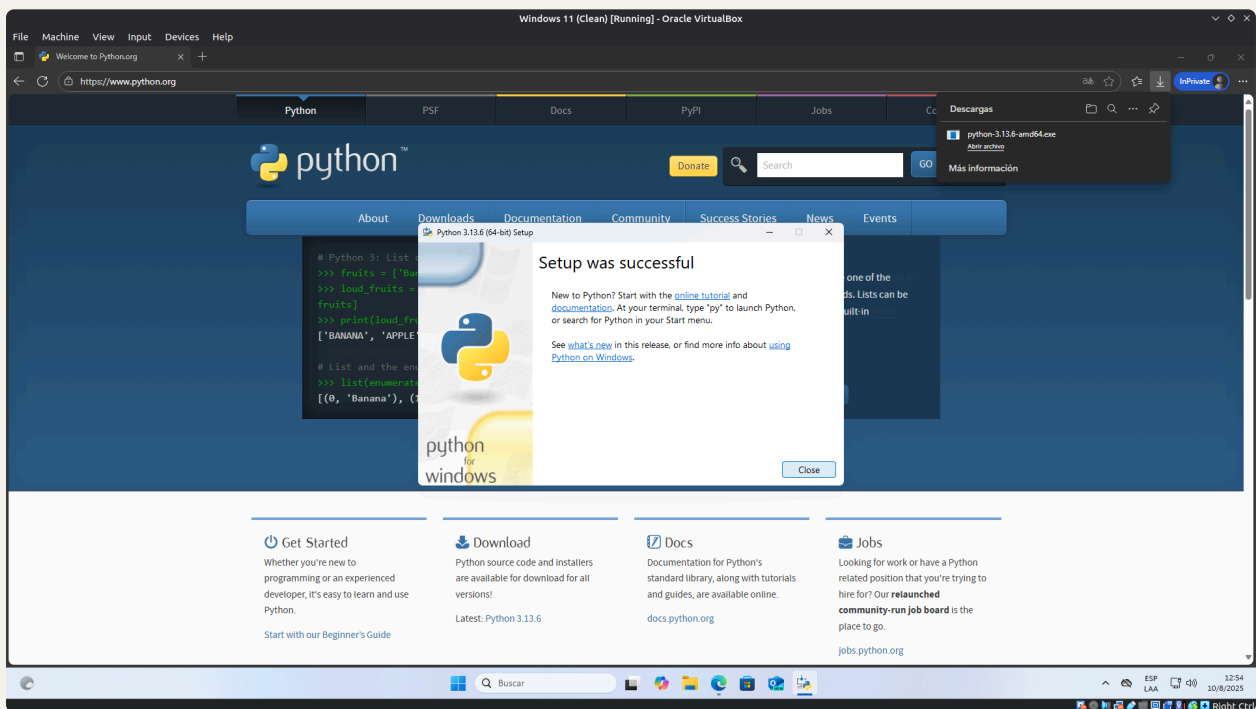
3. Una vez terminada la instalación es importante tocar el botón que dice **«Disable path length limit»**. Esto es para evitar problemas con rutas largas en Windows.



4. Se les va a abrir una ventana pidiendo permisos de administrador, hagan clic en «Sí».



5. Una vez hecho esto, pueden cerrar el instalador apretando el botón «Close».



- Si quieren verificar que Python se haya instalado correctamente abriendo una terminal y escribiendo el comando `python --version`. Deberían ver la versión de Python que instalaron.

Windows Subsystem for Linux (avanzado)

Si ya realizaste la instalación usando el instalador oficial de Python (opción de arriba) o Chocolatey (opción de abajo), no es necesario que sigas este método.

Si desean utilizar Haskell en un entorno Linux, pueden instalar el subsistema de Windows para Linux (WSL). Para ello, deben seguir las instrucciones de la [documentación oficial de Microsoft](#). Luego, deberán ir al apartado de [Linux](#) para seguir con la instalación.

Chocolatey

Si ya realizaste la instalación usando WSL (segunda opción) o el instalador oficial de Python (primera opción), no es necesario que sigas este método.

Deberán instalar el gestor de paquetes Chocolatey, para ello entren a [este link](#) y sigan las instrucciones de la página.

Recuerden que deben seleccionar la opción que dice «**Individual**».

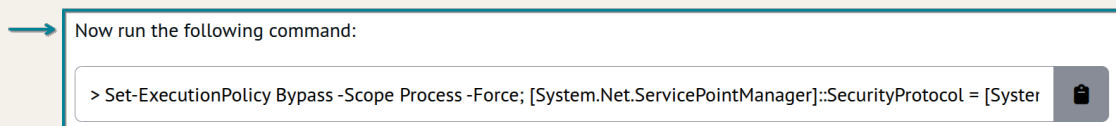
1. Choose How to Install Chocolatey:



Luego, copian y pegan el comando que les aparece en la página en **PowerShell**. Para copiarlo deben hacer clic en el botón que aparece a la derecha del comando, botón que contiene el ícono de un portapapeles.

Para abrir **PowerShell** deben hacer lo siguiente:

1. Aprietan la tecla de Windows.
2. Escriben **PowerShell**.
3. Hacen clic derecho en la aplicación de nombre **PowerShell** que les aparece y seleccionan «**Ejecutar como administrador**».
4. Pegan el comando que **copiaron de la página** y aprietan Enter.



Una vez que se haya instalado Chocolatey, deben cerrar esa terminal de **PowerShell**; acto seguido abrir uno nuevo y ejecutar `choco install python -y`.

2. Linux

El mundo de las distribuciones de Linux es **amplio y diverso**, y **la instalación de Python puede variar según la distribución que estén utilizando**.

Les mostramos cómo instalar Python en algunas de las distribuciones más populares. Para cada una de las distintas distribuciones deberán primero abrir una terminal, para luego, seguir los pasos correspondientes.

Distribuciones basadas en Debian (Ubuntu, Linux Mint, etc.)

1. Actualicen la caché de sus paquetes:

```
sudo apt update
```

2. Instalen Python3 y pip (el gestor de paquetes de Python):

```
sudo apt install python3 python3-pip
```

Distribuciones basadas en Fedora (Red Hat Enterprise Linux, etc.)

1. Actualicen la caché de sus paquetes:

```
sudo dnf update
```

2. Instalen Python3 y pip (el gestor de paquetes de Python):

```
sudo dnf install python3 python3-pip
```

Distribuciones basadas en Arch Linux (EndeavourOS, CachyOS, etc.)

1. Actualicen tanto la caché de sus paquetes como los paquetes instalados:

```
sudo pacman -Syu
```

2. Instalen Python3 y pip (el gestor de paquetes de Python):

```
sudo pacman -S python python-pip
```

3. macOS

Deben instalar [Homebrew](#), un gestor de paquetes para macOS, que les permitirá instalar Python fácilmente. Para instalar [Homebrew](#), abran una terminal y ejecuten el siguiente comando:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Luego, para instalar Python, ejecuten:

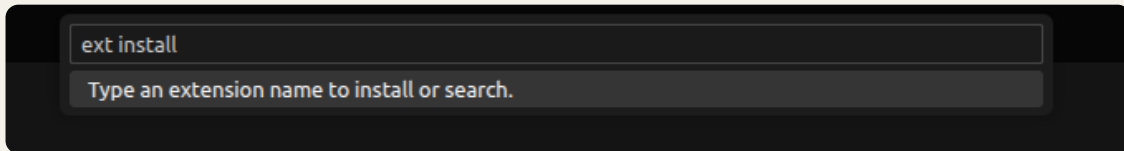
```
brew install python
```

4. Extensiones para Visual Studio Code

Una vez instalado Python, nos va a interesar instalar algunas extensiones a Visual Studio Code que nos ayudarán a mejorar nuestra experiencia de desarrollo.

Para ello, deben abrir Visual Studio Code y seguir estos pasos:

1. Aprieten `Ctrl + P` en Windows/Linux o `Command + P` en macOS para abrir la barra de comandos.



2. Escriban `ext install ms-python.python` y presionen `Enter` para instalar la extensión oficial de Python. Cuando la instalación termine aparecerá en la pestaña de extensiones. Tengan en cuenta que es posible que haya que «recargar» (cerrar y volver a abrir el Visual Studio Code) cuando se termina de instalar, nos lo indicará en la ventana de la extensión.

Si quieren buscar extensiones adicionales, pueden abrir la vista de extensiones haciendo clic en el ícono de extensiones en la barra lateral izquierda o presionando `Ctrl + Shift + X` en Windows/Linux o `Command + Shift + X` en macOS. Desde allí, pueden buscar e instalar otras extensiones que les interesen. Pero tengan cuidado, la tienda de extensiones de Visual Studio Code no es *curada*, es decir, no todas las extensiones están auditadas; descargar una extensión de una fuente no confiable puede comprometer la seguridad de su sistema. Por lo tanto, asegúrense de instalar solo extensiones de fuentes confiables y revisadas por la comunidad.

5. Uso de bibliotecas en un proyecto

Distintos desarrolladores de software han creado bibliotecas³ que pueden ser utilizadas en nuestros proyectos de Python. Estas bibliotecas nos permiten aprovechar el trabajo de otros y enfocarnos en la lógica de nuestro programa.

Particularmente en Python el manejo de bibliotecas es... complejo. Dejamos una guía de cómo instalar y usar bibliotecas en Python, dependiendo del escenario.

pip

`pip` es el gestor de paquetes oficial de Python. Nos permite instalar, actualizar y desinstalar bibliotecas desde la [Python Package Index \(PyPI\)](#), que es el repositorio oficial de paquetes de Python. Veamos que pueden hacer:

- Instalar una biblioteca, por ejemplo, `coverage` :

```
pip install coverage
```

- Actualizar su biblioteca:

```
pip install --upgrade coverage
```

- Desinstalar su biblioteca:

```
pip uninstall coverage
```

Importante para usuarios de Linux:

- Si instalaron Python usando el gestor de paquetes de su distribución (por ejemplo `apt` en Ubuntu), es posible que `pip` no funcione como esperen o que el sistema priorice las bibliotecas instaladas con el gestor de paquetes antes que las instaladas con `pip`. En esos casos, la solución recomendada es crear un entorno virtual (`venv`) y usar `pip` dentro de él.

Nota: en algunos sistemas, dependiendo de su instalación de `pip`, es posible que tengan que usar `pip3` en lugar de `pip`.

³Notar que «librerías» está mal dicho, la confusión viene de traducir erróneamente «library»/«libraries» del inglés.

Entornos virtuales (venv) (recomendado)

Un **entorno virtual** en Python proporciona un espacio aislado con su propio intérprete de Python y bibliotecas, lo que permite gestionar dependencias sin afectar el sistema global. Esto es útil porque distintos proyectos pueden necesitar versiones diferentes de las mismas bibliotecas, y un entorno virtual evita que se mezclen.

Cuando creamos un **entorno virtual**, Python genera una carpeta con el nombre que le indiquemos (por ejemplo venv o entorno) en el lugar donde ejecutamos el comando. Esa carpeta contiene:

- Un intérprete de Python específico para ese entorno.⁴
- Directorios separados para dependencias sin afectar el sistema global.
- Scripts de activación y desactivación del entorno.

Para crear y usar un entorno virtual, deben seguir los siguientes pasos:

1. Ejecuten el siguiente comando:

```
python -m venv nombre_de_su_entorno
```

Observen que se va a crear un directorio con el nombre `nombre_de_su_entorno` dentro del directorio donde están parados cuando ejecuten el comando. Si no desean eso, en el parametro `nombre_de_su_entorno` pongan una ruta absoluta o relativa a donde quieren que se cree el entorno virtual. Ejemplo:

`~/Documentos/ip-venv` para crear un entorno virtual llamado `ip-venv` dentro de la carpeta `Documentos` de su usuario en Linux/macOS.

Nota: en algunos sistemas, dependiendo de su instalación de Python, es posible que tengan que usar `python3` en lugar de `python`.

2. Activen el entorno ejecutando:

- En Windows (utilizando PowerShell)
`ruta_absoluta_de_su_entorno\Scripts\Activate.ps1`
- En Windows (utilizando cmd):
`ruta_absoluta_de_su_entorno\Scripts\activate`
- En Linux/macOS (utilizando tanto bash como zsh):
`source ruta_absoluta_de_su_entorno/bin/activate`

Mientras el entorno esté activado, las bibliotecas que instalen con `pip` solo se agregarán allí, sin afectar al resto del sistema.

3. Una vez que no lo queramos usar más, podemos desactivarlo ejecutando:

```
deactivate
```

¿Por qué es preferible usar `pip` en un `entorno virtual` a usar `pip` globalmente?:

- **Evita conflictos** entre proyectos.
- **Funciona igual en Windows, Linux y macOS.**
- **Permite instalar bibliotecas sin necesidad de permisos de administrador**, por ejemplo en los laboratorios de la facultad, donde no es posible usar `apt` para instalar paquetes globales porque se requieren privilegios de administrador.

⁴En realidad son varios [enlaces simbólicos](#), pero no es necesario entrar en detalles técnicos.

Bibliotecas distribuidas por quienes mantienen la distribución de Linux

En las distribuciones de Linux existen gestores de paquetes (por ejemplo: `apt`, `dnf`, `pacman`) que permiten instalar software, incluidas bibliotecas de Python.

Si instalaron Python a través del gestor de paquetes de su distribución, es posible que:

- El sistema detecte conflictos entre las bibliotecas del gestor de paquetes y las de `pip`.
- `pip` te sugiera muy fuertemente usar el gestor del sistema en lugar de instalar la biblioteca.

En estos casos, muchas bibliotecas de **PyPI** tienen su equivalente como paquete del sistema. Por ejemplo, en lugar de instalar `coverage` con `pip install coverage` pueden instalarlo con `sudo apt install python3-coverage` en Debian/Ubuntu/Mint o `sudo pacman -S python-coverage` en Arch Linux.

Esto puede ser útil si:

- Quieren instalar algo de forma global para todos los usuarios del sistema.
- La biblioteca necesita dependencias del sistema y la distribución ya preparó el paquete para que funcione.

¿Nuestra recomendación? Aunque el gestor de paquetes pueda instalar la biblioteca, siempre es más seguro y flexible **usar un entorno virtual** y `pip` dentro de él. De esta forma evitan conflictos de versiones y aseguran que tu proyecto funcione igual en cualquier computadora, sin depender de cómo esté configurado el sistema.

Troubleshooting

En caso de que se encuentren con algún problema, pueden consultar esta sección.

1. Me olvidé de agregar python.exe al PATH en Windows

Tienen dos opciones para solucionar esto, una reinstalar Python asegurándose de marcar la opción de agregar Python al PATH.

La otra es agregar ustedes mismos las rutas de Python al PATH de su usuario. Para esto, deben crear un archivo llamado `Agregar_Python_al_PATH.ps1` con lo siguiente:

```
# Ruta base donde normalmente se instala Python
$pythonBase = "$env:USERPROFILE\AppData\Local\Programs\Python"

# Verificamos si existe la carpeta base
if (-Not (Test-Path $pythonBase)) {
    Write-Host "No se encontró la carpeta base de Python: $pythonBase"
    exit 1
}

# Buscamos carpetas que coincidan con PythonXXX (por ejemplo: Python312, Python313)
$pythonDirs = Get-ChildItem -Path $pythonBase -Directory |
    Where-Object { $_.Name -match "^Python\d{3}$" } |
    Sort-Object Name -Descending

if ($pythonDirs.Count -eq 0) {
    Write-Host "No se encontraron instalaciones de Python en: $pythonBase"
    exit 1
}

# Tomamos la versión más reciente encontrada
$ultimaVersion = $pythonDirs[0].Name
$pythonPath = Join-Path $pythonBase $ultimaVersion
$scriptsPath = Join-Path $pythonPath "Scripts"

# Listamos las rutas que se van a agregar al PATH del usuario
$rutasAAgregar = @(
    "$pythonBase\Launcher\",
    $pythonPath,
    $scriptsPath
)

# Obtenemos el PATH actual del usuario
$pathExistente = [Environment]::GetEnvironmentVariable("Path", "User")

# Verificamos si las rutas ya están en el PATH
$rutasExistentes = $pathExistente -split ';'

# Filtramos las rutas a agregar, eliminando las que ya están presentes
$rutasNoDuplicadas = $rutasAAgregar | Where-Object { $_ -notin $rutasExistentes }

if ($rutasNoDuplicadas.Count -eq 0) {
    Write-Host "Las rutas ya están agregadas al PATH del usuario."
} else {
    # Creamos el nuevo PATH
    $nuevoPath = ($rutasNoDuplicadas -join ';') + ";" + $pathExistente
    # Establecemos el nuevo PATH para el usuario
    [Environment]::SetEnvironmentVariable("Path", $nuevoPath, "User")
    Write-Host "El PATH del usuario fue actualizado con éxito. Se agregaron las siguientes rutas:"
    $rutasNoDuplicadas | ForEach-Object { Write-Host "    • $_" }
}
```

Luego, deben abrir PowerShell y ejecutar

```
Set-ExecutionPolicy Bypass -Scope Process -Force; .\Agregar_Python_al_PATH.ps1 .
```

2. Me olvidé de deslimitar los caracteres para direcciones largas en Windows

Deben crear un archivo llamado `Deslimitar_caracteres.ps1` con el siguiente contenido:

```
# Verificamos si el script está ejecutándose como Administrador
$esAdministrador =
[System.Security.Principal.WindowsIdentity]::GetCurrent().Groups -match
"S-1-5-32-544"
if (-not $esAdministrador) {
    Write-Host "Este script debe ejecutarse como Administrador."
    exit 1
}

# Ruta del registro donde se desactiva el límite de 260 caracteres
$claveRegistro = "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem"
$nombreValor = "LongPathsEnabled"

# Intentamos modificar el registro para desactivar el límite de 260 caracteres
try {
    Set-ItemProperty -Path $claveRegistro -Name $nombreValor -Value 1 -Type DWord
    -Force
    Write-Host "Se desactivó el límite de 260 caracteres en rutas largas."
} catch {
    Write-Host "No se pudo modificar el registro."
}
```

Luego, deben abrir un PowerShell **como administrador** y ejecutar

```
Set-ExecutionPolicy Bypass -Scope Process -Force; .\Deslimitar_caracteres.ps1 .
```


3. No tengo una computadora disponible donde instalar lo necesario para realizar la materia

Hay varias opciones que se pueden considerar para realizar la materia. Lo primero es que siempre están los laboratorios de la facultad, aunque es entendible que no siempre se pueda ir a la facultad.

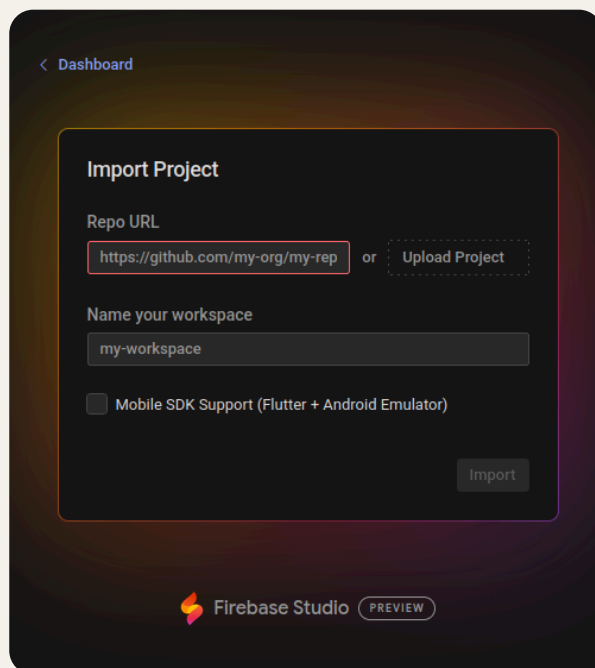
Es por ello que les presentamos una alternativa para que puedan programar desde cualquier dispositivo con acceso a internet (inclusive sus celulares).

1. Deben crearse una cuenta en [GitLab](#). Vamos a suponer que su usuario se llama **«intro-prog»**.
2. Con la cuenta creada y la sesión iniciada, deben ir al [repositorio que preparamos](#) y buscar el botón que dice **«Fork»** para crear una copia del repositorio en su cuenta.



Project information

3. Con el fork creado, deberían tener un repositorio llamado **«ip-idx»**, con el enlace <https://gitlab.com/intro-prog/ip-idx> (donde **intro-prog** es su usuario). Lo van a copiar, para luego ir a la sección de [importar repositorios en Firebase Studio](#), donde deben iniciar sesión con su cuenta de Google. Una vez que hayan iniciado sesión, verán una pantalla como la de la imagen.
4. Deben pegar el enlace del repositorio que copiaron antes donde dice **«Repo URL»** y luego hacer clic en **«Import»**. Tras importar se va a empezar a construir el entorno para ustedes, les **va tardar un rato medianamente largo**, aunque **Firebase Studio** les pregunte si quieren saltar la construcción, no lo hagan, **esperen pacientemente**.
5. Una vez que Firebase Studio **termine de construir el entorno**, van a tener un **entorno basado en Visual Studio Code** listo con: los **lenguajes de programación**, las **extensiones** y las **bibliotecas** necesarias para la materia. Además, pueden utilizar la **terminal integrada** de la interfaz web de Firebase Studio para ejecutar los comandos que necesiten.



Glosario

- **Terminal / Consola / Símbolo del sistema / Línea de comandos:** programa para interactuar con la computadora⁵, escribiendo comandos de texto.
- **Comando:** instrucción escrita en la terminal para que la computadora ejecute una acción.
- **Ruta / Path:** dirección que indica la ubicación de un archivo o carpeta.
 - **Ruta absoluta:** dirección completa que indica la ubicación exacta de un archivo o carpeta desde la raíz del sistema.
 - **Ruta relativa:** dirección de un archivo o carpeta escrita en relación a la ubicación actual (directorio de trabajo) o algún otro punto de referencia (como la carpeta personal).
- **Directorio / Carpeta:** lugar donde se guardan archivos y otros directorios.
- **Home / Carpeta personal:** carpeta principal de cada usuario del sistema.
- **Biblioteca:** conjunto de funciones y herramientas listas para usar en un lenguaje de programación.⁶
- **pip:** gestor de paquetes de Python que permite instalar bibliotecas desde internet.⁷
- **PyPI:** repositorio oficial donde están publicadas las bibliotecas de Python.
- **Entorno virtual / venv:** carpeta que contiene una instalación aislada de Python con sus bibliotecas.
- **Gestor de paquetes:** programa que instala, actualiza y elimina software/ bibliotecas; `apt` (Debian/Ubuntu/Mint), `dnf` (Fedora), `pacman` (Arch Linux), `pip` (Python) y `npm` (Node.js) son algunos ejemplos.
- **Compilador:** programa que traduce código fuente a un lenguaje que la computadora pueda ejecutar.
- **Intérprete:** programa que lee y ejecuta el código directamente, sin compilarlo previamente.⁸
- **Variable de entorno:** valor que el sistema guarda y que puede afectar el comportamiento de programas.
- **Permisos de administrador:** autorización especial para realizar cambios que afectan a todo el sistema.
- **Repositorio:** lugar donde se almacenan programas o bibliotecas para descargarlos.
- **Dependencia:** otro programa o biblioteca que necesita un software para funcionar.
- **Haskell:** lenguaje de programación funcional de tipado estático y fuerte, utiliza evaluación lazy.
- **GHC (Glasgow Haskell Compiler):** un compilador para Haskell.
- **GHCi:** un intérprete para Haskell.
- **Python:** lenguaje de programación imperativo de tipado dinámico y fuerte.
- **Script:** archivo que contiene instrucciones de código que se ejecutan en un lenguaje de programación.

⁵Formalmente se le dice “la shell”.

⁶Se le suele llamar erróneamente «librería».

⁷pip es el acrónimo de «pip installs packages». Se trata de un [acrónimo recursivo](#), una convención común en el ámbito del software.

⁸En un sentido más teórico, también se llama intérprete a un mecanismo (a veces dentro de un compilador) que describe cómo se evalúan las expresiones según ciertas reglas.

- **Shell:** programa que interpreta comandos escritos por el usuario y los ejecuta en el sistema operativo.
- **Bash:** un intérprete de comandos muy utilizado en sistemas Unix/Linux, que permite ejecutar comandos y scripts; es el predeterminado en muchas distribuciones de Linux.
- **Zsh:** un intérprete de comandos similar a Bash, pero con más características y personalización; es el predeterminado en macOS.
- **PowerShell:** intérprete de comandos avanzado de Windows, que permite ejecutar scripts y administrar el sistema de manera más potente que el CMD tradicional.⁹
- **cmd (Command Prompt):** intérprete de comandos clásico de Windows (un poco discontinuado).
- **PATH:** variable de entorno que indica al sistema en qué carpetas buscar programas para ejecutarlos.
- **Compilar:** proceso de traducir código fuente a un formato ejecutable.
- **Ejecutable:** archivo que el sistema puede correr directamente.
- **Repositorio (Git):** lugar donde se almacena el código fuente de un proyecto, junto con su historial de cambios.
- **Git:** sistema de control de versiones que permite llevar un registro de los cambios realizados en archivos y coordinar el trabajo entre varias personas.
- **GitLab:** plataforma para alojar repositorios Git en la nube, colaborar en proyectos y gestionar versiones.
- **Binario:** archivo que contiene código ejecutable en formato máquina.
- **Cabal:** gestor de paquetes (y herramienta de construcción de proyectos) para Haskell, al igual que `pip`, facilita la instalación de bibliotecas y la gestión de dependencias.
- **Caché de paquetes:** espacio donde el sistema guarda temporalmente los datos descargados de los repositorios, para que las instalaciones y actualizaciones sean más rápidas.
- **Distribución (Linux):** variante de un sistema operativo basado en Linux, que incluye su propio conjunto de programas, configuraciones y repositorios.
- **Fork:** copia de un proyecto existente en Git que permite modificarlo sin afectar al original, creando una nueva versión independiente.
- **Firebase Studio:** herramienta online de Google que permite programar y ejecutar código en un entorno de desarrollo basado en navegador, sin necesidad de instalar software en la computadora local (utiliza una de Google).
- **WSL (Windows Subsystem for Linux):** funcionalidad de Windows que permite instalar y usar un sistema Linux dentro de Windows, sin necesidad de usar una máquina virtual o reiniciar el sistema.

⁹También está «PowerShell», la aplicación gráfica que permite ejecutar comandos de PowerShell en una ventana. Aquí estamos hablando de PowerShell como una shell.